

Laboratorio 4 — Regresión lineal y clasificación

Luis Seña Ávila, Universidad de Antioquia

Medellín - Antioquia

21 de septiembre de 2026

Resumen

En este laboratorio se implementaron los modelos de regresión lineal y logística, siguiendo los cinco pasos: hipótesis, función de costo, gradiente, optimizador y evaluación. Se trabajó con datos sintéticos y reales (diabetes e Iris), comprobando la importancia del escalamiento de características, la eficacia de la ecuación normal para pocas variables y el efecto de la regularización para controlar el sobreajuste. Los resultados propios coincidieron con los de Scikit-learn, validando cada implementación.

Palabras Claves: Regresión lineal, regresión logística, gradiente descendente, ecuación normal, regularización.

1. Introducción

La regresión lineal y la regresión logística son dos de los modelos más fundamentales en machine learning. La primera se usa para predecir valores continuos (como precios o temperaturas), mientras que la segunda se usa para clasificar (como decidir si un correo es spam o no). Aunque parecen distintos, ambos comparten la misma estructura: una hipótesis, una función de costo y un gradiente que se optimiza para encontrar los mejores parámetros.

En este informe se presenta el desarrollo del laboratorio en el siguiente orden. En la Sección 2.1 se explora la regresión lineal más simple, con una sola característica, entendiendo cómo se mide el error mediante la función de costo. Luego, en la Sección 2.2, se implementa el gradiente descendente, el algoritmo que permite minimizar ese costo paso a paso. Después, en la Sección 2.3, se generaliza a múltiples características usando álgebra matricial, y en la Sección 2.4 se aborda el problema del escalamiento de los datos. Más adelante, en la Sección 2.5, se presenta la ecuación normal, una solución cerrada que evita iterar. Finalmente, en las Secciones 2.6 y 2.7, se introduce la regresión logística para clasificación y se discute la regularización para evitar el sobreajuste. En la Sección 3 se responden las preguntas de análisis, y en la Sección 4 se cierra con las conclusiones.

2. Desarrollo

2.1. Regresión Lineal: Hipótesis y Costo

Se generaron datos sintéticos con $\theta_0 = 0,2$ y $\theta_1 = 0,5$. La función de costo J se minimizó en $\theta_1 \approx 0,495$, confirmando que el barrido funciona. La superficie de costo es un paraboloide convexo con un único mínimo.

2.2. Gradiente Descendente

Se implementó el descenso con $\alpha = 0,5$, convergiendo a valores cercanos a los reales. Al probar $\alpha = 3,0$ el costo divergió (se fue a infinito), y con $\alpha = 0,01$ avanzó muy lento. Esto muestra que la tasa de aprendizaje es crítica.

2.3. Regresión Multivariada

Con dos características se usó la forma matricial: $h = X\Theta$, $J = \frac{1}{2m}(h - y)^T(h - y)$ y $\nabla_{\Theta}J = \frac{1}{m}X^T(h - y)$. El gradiente vectorizado se verificó contra una derivada numérica, obteniendo coincidencia exacta.

2.4. Escalamiento de Características

En esta sección se trabajó con dos características que vivían en escalas muy distintas: z_1 en $[-1, 1]$ y z_2 en $[-2000, 2000]$, una diferencia de 2000 veces. Se generaron 100 datos con la relación $y = 2z_1 - 3z_2$ + ruido.

El primer experimento fue entrenar sin escalar con $\alpha = 0,3$. El costo explotó inmediatamente: en la primera iteración fue $9,4 \times 10^{17}$, luego $1,4 \times 10^{29}$, y tras 30 iteraciones divergió a infinito. Esto pasa porque el gradiente en la dirección de z_2 es gigante, y un α que serviría para z_1 es demasiado grande para z_2 .

El segundo experimento fue reducir el α a 1×10^{-8} durante 2000 iteraciones. El costo ya no divergió, pero el resultado fue $\Theta = [0, 0, -3]$, es decir, el coeficiente de z_1 se quedó en cero mientras el de z_2 sí llegó al valor real. La razón es que el gradiente de z_1 es diminuto y con un paso tan pequeño nunca avanza.

La solución fue estandarizar: a cada característica se le restó su media y se dividió por su desviación estándar, dejando ambas en una escala comparable. Con los datos escalados y $\alpha = 0,3$, el costo final fue 0,0041, convergiendo correctamente. Al devolver los coeficientes al espacio original, se obtuvo $[0,006, 1,987, -3]$, muy cercano al real $[0, 2, -3]$. La conclusión es clara: sin escalar, el gradiente descendente no funciona bien; con

escalamiento, un solo α sirve para todas las direcciones.

2.5. Ecuación Normal

Se implementó $\Theta = (X^T X)^{-1} X^T y$ usando `np.linalg.solve`. Los resultados coincidieron con el gradiente descendente y con Scikit-learn (diferencia máxima de $1,7 \times 10^{-12}$). En el dataset de diabetes, el R^2 fue 0,5177.

2.6. Clasificación: Regresión Logística

Se usó la sigmoide y entropía cruzada. El gradiente resultó ser $\frac{1}{m} X^T (h - y)$, idéntico en forma al de regresión lineal. El clasificador propio alcanzó una exactitud de 0,975, igual que Scikit-learn.

2.7. Multiclase y Regularización

Se usó regresión logística multinomial con el dataset Iris. El barrido de C (inverso de λ) mostró que valores muy pequeños causan subajuste y muy grandes sobreajuste. El mejor valor fue $C = 1$ con exactitud de prueba 0,911.

3. Respuestas a las preguntas de análisis

1. ¿Por qué el error cuadrático produce una superficie de coste convexa mientras que aplicado a la sigmoide no lo hace?

Porque el error cuadrático es una función cuadrática de Θ , lo que genera un paraboloide (convexo). Al meter la sigmoide, la función se vuelve no lineal y aparecen múltiples mínimos locales. Por eso en clasificación se usa entropía cruzada, que sí es convexa.

2. Usted entrena con $\alpha = 0,01$ durante 100 iteraciones y el coste baja muy poco. Enumere dos causas posibles y cómo distinguirlas.

- **Alpha muy pequeño:** El costo baja suavemente pero lento. Se distingue porque al aumentar α (ej. 0.3) el costo baja mucho más rápido.
- **Características sin escalar:** La superficie es un valle alargado. Se distingue porque un coeficiente se mueve mucho y el otro casi nada. La solución es estandarizar.

3. Con $n = 50,000$ características, ¿usaría la ecuación normal? Justifique con la tabla de la Parte 5.

No. La ecuación normal tiene costo $O(n^3)$ por la inversión de $(X^T X)$, que con 50,000 características es inviable. El gradiente descendente tiene costo $O(kn^2)$, mucho más manejable. La tabla de la Parte 5 muestra que la ecuación normal solo sirve para n pequeño.

4. En la gráfica del Ejercicio 7.1, ¿qué región corresponde a subajuste y cuál a sobreajuste? ¿Qué valor de C reportaría y por qué?

- **Subajuste:** C muy pequeño (mucha regularización). El modelo es demasiado simple y falla en train y test.

- **Sobreajuste:** C muy grande (poca regularización). El modelo memoriza train pero falla en test.

- **Valor a reportar:** $C = 1$, porque maximiza la exactitud en prueba, logrando el mejor balance entre sesgo y varianza.

4. Conclusiones

- **El gradiente descendente depende críticamente del α :** con $\alpha = 0,5$ convergió bien, con $\alpha = 3,0$ divergió a infinito, y con $\alpha = 0,01$ avanzó demasiado lento. No hay un valor universal, hay que ajustarlo.
- **Sin escalamiento, el gradiente descendente no sirve:** una característica en $[-2000, 2000]$ junto a otra en $[-1, 1]$ hizo divergir el costo con $\alpha = 0,3$, y con $\alpha = 1 \times 10^{-8}$ solo aprendió la variable grande. Al estandarizar, un solo α funcionó para ambas.
- **La ecuación normal es exacta pero costosa:** coincidió con el gradiente descendente y con Scikit-learn (diferencia de $1,7 \times 10^{-12}$), pero su costo $O(n^3)$ la hace inviable para muchas características. Sirve para n pequeño.
- **El gradiente de la regresión logística es idéntico al de la lineal:** solo cambia la definición de h (sigmoide en vez de producto punto). Esto permitió reutilizar la misma estructura de código.
- **La entropía cruzada es la función de costo correcta para clasificación:** el error cuadrático con sigmoide genera una superficie no convexa con múltiples mínimos locales.
- **La regularización controla el sobreajuste:** valores muy pequeños de C causan subajuste, valores muy grandes sobreajuste, y el mejor balance se encontró en $C = 1$ con exactitud de prueba de 0,911.
- **La implementación propia fue validada:** tanto en regresión lineal (contra ecuación normal y Scikit-learn) como en regresión logística (contra Scikit-learn), los resultados coincidieron, confirmando que los cálculos manuales son correctos.

5. Bibliografía

6. Bibliografía

Referencias

- [1] Stanford CS229 — Machine Learning. <http://cs229.stanford.edu/syllabus.html>
- [2] A. Ng. *Machine Learning* (Coursera), semanas 1–3.

- [3] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, cap. 4. O'Reilly.
- [4] Scikit-learn: Machine Learning in Python. <https://scikit-learn.org/stable/>

El notebook completo con el desarrollo, los experimentos y los resultados puede consultarse en el siguiente enlace: https://colab.research.google.com/drive/1ectYHD_OqhwmTNZDF1qnB8EnMohU4NMX?usp=sharing

Declaración de uso de inteligencia artificial

Para la elaboración de este informe se utilizó la herramienta de inteligencia artificial DeepSeek (modelo de lenguaje) como apoyo en las siguientes tareas:

- Redacción y mejora de la claridad de las explicaciones en las secciones de desarrollo.
- Verificación de la coherencia entre los resultados del notebook y las conclusiones presentadas.
- Apoyo en la redacción de las respuestas a las preguntas de análisis.

El análisis de los datos, la implementación de los algoritmos, la ejecución del notebook y la interpretación de los resultados fueron realizados por el mi. Las sugerencias proporcionadas por la IA fueron revisadas, validadas y adaptadas antes de su inclusión en el documento. El uso de esta herramienta se declara en cumplimiento de los principios de transparencia y honestidad académica.